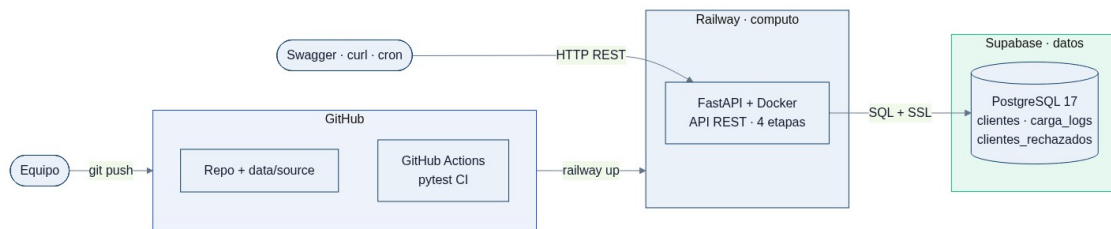


DUOC UC

Escuela de Informática y Telecomunicaciones

Informe Técnico — Evaluación Parcial N°2

Pipeline DataOps para Predicción de Churn en Telecomunicaciones



Asignatura	ITY1101 — Gestión de Datos para IA
Sección	003V
Caso	Telco Customer Churn (clasificación de abandono)
Integrantes	Benjamín Heresmann — Ingeniero de Datos (ingesta, validación, arquitectura) Diego Hernández — Ingeniero de BD / DataOps (limpieza, carga, seguridad, deploy)
Arquitectura	Pipeline modular cloud (FastAPI · Railway · Supabase)
Metodología	PMBOK híbrido (predictivo + adaptativo)
Fecha de entrega	2 de junio de 2026

Índice

1. Resumen ejecutivo
2. Justificación de la metodología PMBOK
3. Planificación del proyecto (WBS, hitos, Carta Gantt, seguimiento)
4. Arquitectura cloud desacoplada
5. Explicación técnica del pipeline (4 etapas)
6. Plan de seguridad para entorno DataOps
7. Estrategia de KPIs de monitoreo
8. Documentación, evidencias y modelo de datos
9. Conclusiones y próximos pasos

1. Resumen ejecutivo

Este informe documenta el diseño, planificación e implementación de un pipeline de datos automatizado, bajo principios **DataOps**, para una compañía de telecomunicaciones que enfrenta un alto índice de abandono de clientes (**churn** $\approx 26,5\%$). El proyecto usa el dataset público “Telco Customer Churn” (7.043 clientes, 21 variables) para construir un flujo de cuatro etapas —ingesta, limpieza y transformación, validación estructural y semántica, y carga a base de datos relacional— que deja la información lista para entrenar un modelo predictivo de IA en la Evaluación 3.

El **valor para la organización** es doble: reduce el costo y los errores de la preparación manual de datos, y habilita la focalización de campañas de retención sobre los clientes con mayor probabilidad de abandono, protegiendo los ingresos recurrentes. La solución se implementa con una **arquitectura cloud-native desacoplada**: el procesamiento corre como API REST en **Railway** (cómputo), la persistencia usa **PostgreSQL gestionado en Supabase** (datos), y el código se versiona en **GitHub** con CI automatizado (pytest en cada push). No es un monolito: cada componente escala y se despliega de forma independiente. El sistema está desplegado, probado end-to-end y operativo en producción.

El equipo, autorizado por el docente a operar con **dos integrantes**, dividió las responsabilidades técnicas de forma equitativa manteniendo dominio individual de toda la solución.

2. Justificación de la metodología PMBOK

Se adopta un **enfoque híbrido del PMBOK** (predictivo + adaptativo). El **componente predictivo** se aplica a actividades de requisitos fijos y dependencias claras —modelado de la base de datos, esquema de validación, Dockerfile e integración con el repositorio—, planificadas con cronograma, hitos y entregables. El **componente adaptativo** se aplica a lo exploratorio —afinamiento de reglas de validación semántica, KPIs y ensayo de la demo—, trabajado en ciclos cortos con revisión cruzada entre los integrantes.

Una metodología puramente cascada sería rígida frente a los hallazgos que aparecen al procesar el dataset por primera vez; una puramente ágil desordenaría las dependencias técnicas. El híbrido permite **planificar lo estable e iterar lo incierto**, coherente con un equipo pequeño y un plazo acotado. El seguimiento se realiza con un tablero **Trello** (columnas Por hacer / En progreso / Terminado), elegido sobre Jira o Azure DevOps por simplicidad y escala del equipo.

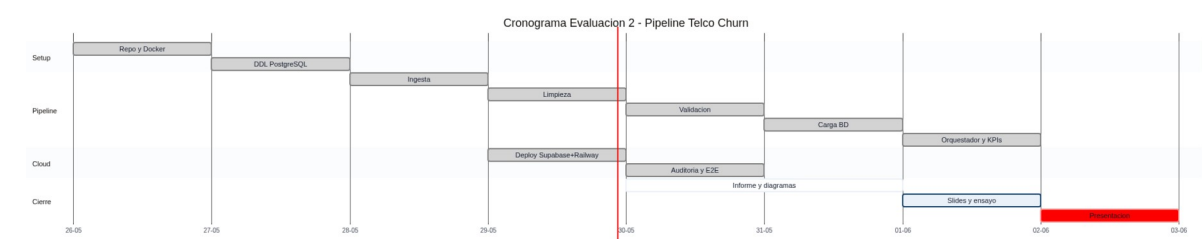
3. Planificación del proyecto

3.1 Estructura de Desglose del Trabajo (WBS) e hitos

Fase	Actividades principales	Entregable
1. Setup	Arquitectura, stack, repo, Docker, DDL	Repo y entorno cloud operativo
2. Pipeline	Ingesta, limpieza, validación, carga, orquestador, KPIs	Pipeline end-to-end funcional
3. Calidad	Tests, auditoría, plan de seguridad	Tests verdes + reporte auditoría
4. Documentación	Informe, diagramas, slides, ensayo demo	Informe PDF + presentación

Hitos: H1 — stack y entorno cloud operativo; H2 — pipeline end-to-end cargando datos en Supabase; H3 — documentación, auditoría y demo listas para la defensa.

3.2 Carta Gantt

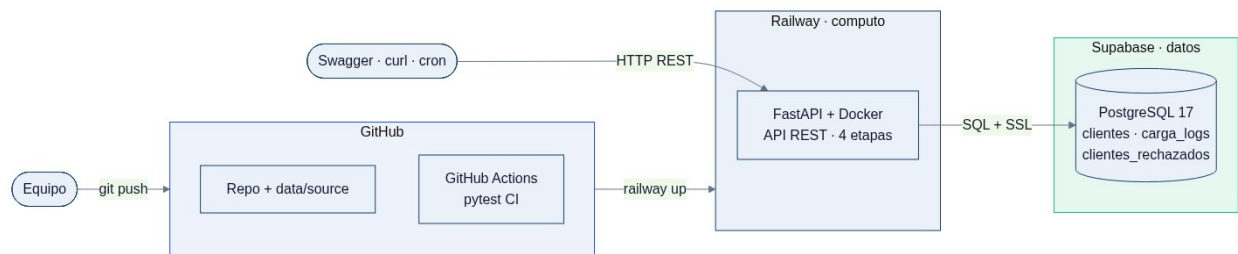


4. Arquitectura cloud desacoplada

A solicitud del docente, la solución **no es un monolito** y queda desplegada en la nube. Se diseñan tres servicios independientes que se comunican por interfaces estándar (HTTPS REST y SQL sobre TLS):

Capa	Servicio	Rol
Datos	Supabase (PostgreSQL 17)	Persiste datos curados, auditoría y rechazados; SSL nativo
Cómputo	Railway (Docker + FastAPI)	Ejecuta el pipeline expuesto como API REST
CI	GitHub Actions	Corre pytest en cada push (gate de calidad)
CD	railway up	Deploy por comando (gate humano; auto-deploy activable)

Por qué desacoplar: cada servicio se actualiza, escala y reinicia por separado; si Railway cae, la BD permanece; si crece el tráfico se escala solo la API; y sustituir un componente (p. ej. cambiar Supabase por otro Postgres) solo implica actualizar la cadena de conexión. Esto reduce el radio de impacto de cualquier fallo frente a un monolito.



La API expone cada etapa como endpoint REST independiente (/pipeline/ingest, /clean, /validate, /load) más un orquestador /pipeline/run, además de monitoreo (/health, /kpis/resumen, /kpis/last, /logs/last, /rechazados) y documentación interactiva autogenerada en /docs (Swagger).

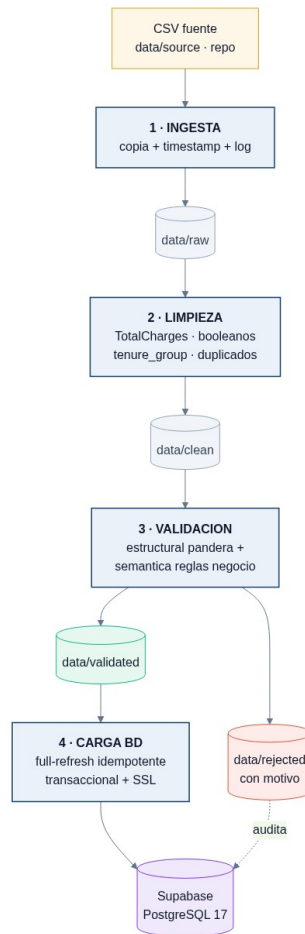
4.1 Escalabilidad del despliegue: un contenedor por capa

Siguiendo el principio de modularidad, **cada etapa del pipeline se empaqueta como un contenedor independiente** (un servicio por capa: ingesta, limpieza, validación y carga), definidos en docker-compose.yml. Así cada capa se **modifica y escala por separado**: se puede replicar solo el contenedor con más carga (p. ej. docker compose up --scale limpieza=3) sin tocar las demás, y un fallo queda aislado en su contenedor. En la versión local las capas

comparten un volumen de datos con las zonas data/source → data/raw → data/clean → data/validated.

Escalado a producción (arquitectura objetivo): cada capa se despliega como su propio contenedor/servicio en **Railway**, y la **salida de cada capa queda en una carpeta de Supabase Storage** (un bucket o prefijo por zona: raw/, clean/, validated/), que es la entrada de la etapa siguiente. Se utiliza Storage en lugar de un volumen local porque los servicios cloud independientes no comparten disco; la orquestación queda a cargo del gateway (la API actual) o de un planificador como Airflow. La demo en vivo corre sobre la API unificada en Railway, que comparte el mismo código e imagen que los contenedores por capa.

5. Explicación técnica del pipeline



5.1 Ingesta

Captura el CSV fuente y lo deposita en data/raw/ con sello temporal, sin transformarlo. Estrategia de fuentes en cascada: parámetro explícito → Supabase Storage → ruta local → **fallback por defecto: dataset versionado en el repo** (data/source/), que viaja dentro de la imagen Docker. Se elige **ingesta por lotes (batch)** porque el dato es un CSV estático; streaming/Kafka sería sobreingeniería. El dataset versionado hace la ingesta reproducible y sin dependencias externas en la demo.

5.2 Limpieza y transformación

Normaliza y enriquece sin aplicar reglas de negocio: (1) convierte TotalCharges de texto con celdas vacías a numérico —resolviendo el bug clásico del dataset—, imputando 0 a los 11 clientes con tenure=0 (recién registrados, sin facturación, imputación justificada); (2) convierte

columnas Yes/No a booleano; (3) elimina duplicados por customerID; (4) crea la feature derivada tenure_group (5 rangos). Se trabaja sobre copia, nunca sobre el crudo.

5.3 Validación estructural y semántica

Estructural (pandera): tipos por columna, rangos (tenure 0–100, MonthlyCharges ≥ 0), valores permitidos en categóricas y formato de customerID por expresión regular. **Semántica (reglas de negocio):** coherencia entre campos —si InternetService=No, los seis servicios derivados deben ser “No internet service”; si PhoneService=False, MultipleLines debe ser “No phone service”; coherencia de TotalCharges con MonthlyCharges*tenure. Las filas válidas pasan a data/validated/; las inválidas a data/rejected/ con su **motivo**, y se auditan en la tabla clientes_rechazados. Se eligió pandera sobre Great Expectations por simplicidad e integración con pandas.

5.4 Carga a base de datos

Persiste los validados en Supabase con SQLAlchemy + psycopg2 y SSL. La carga es **full-refresh idempotente**: dentro de una transacción hace TRUNCATE de las tablas de estado y luego inserta, de modo que reejecutar el pipeline produzca siempre el mismo resultado (reproducibilidad DataOps), sin errores de clave duplicada. La auditoría (carga_logs) **nunca** se trunca. La operación es atómica: ante fallo, ROLLBACK deja la tabla intacta. El engine se cachea (singleton con pool acotado) para no agotar el pooler de Supabase.

Manejo de anomalías: errores de conexión → estado “ERROR” registrado en carga_logs; violación de restricción → rollback; datos inválidos → separados y auditados, nunca descartados silenciosamente.

6. Plan de seguridad para entorno DataOps

El pipeline maneja datos personales bajo el alcance de la **Ley 19.628** (Protección de la Vida Privada, Chile) y la **Ley 21.459** (delitos informáticos), con controles compatibles con ISO/IEC 27001.

Ámbito	Medida implementada
Normas legales	Ley 19.628 (datos personales) y Ley 21.459; tratamiento limitado al propósito declarado
Cifrado en tránsito	Conexión a PostgreSQL forzada con SSL (sslmode=require)
Cifrado en reposo	Volumen gestionado por Supabase; opción pgcrypto a nivel columna
Control de acceso	Rol telco_analista solo-SELECT (mínimo privilegio); credenciales por rol
Gestión de secretos	Variables de entorno en Railway y .env gitignored; cero credenciales en el repo
Enmascaramiento	customerID anonimizado; hashing SHA-256 con sal para datos identificables
Anti-inyección	SQLAlchemy con parámetros bindados; sin concatenación de SQL
Logs sin PII	Se registran conteos y tipos de error, no valores de columnas sensibles
Auditoría	Tabla carga_logs (quién/cuándo/cuántos/estado) + clientes_rechazados

7. Estrategia de KPIs de monitoreo

Cada ejecución mide y persiste indicadores en carga_logs, consultables vía /kpis/resumen y /kpis/last. Si un KPI cruza su umbral, el logger emite WARNING (en producción, integrable a Grafana/alertas):

KPI	Definición	Umbral de alerta
Latencia total	Segundos del pipeline completo	> 30 s
Latencia por etapa	Segundos por etapa	> 50% del total
Tasa de validez	% de registros que pasan validación	< 95%
Compleitud	% de no nulos por columna crítica	< 95%
Volumen	Registros procesados	< 5.000

KPI	Definición	Umbral de alerta
Tasa de error en carga	% inserts fallidos / estado de ejecución	≠ OK

8. Documentación, evidencias y modelo de datos

8.1 Recursos en producción

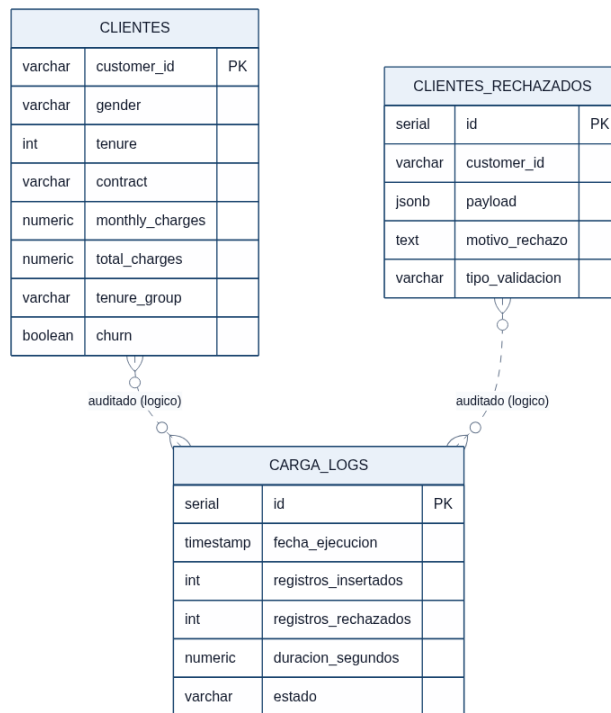
Recurso	Enlace
Repositorio GitHub	github.com/BenjaminHeresmann/telco-churn-pipeline
API en producción	telco-api-production-e466.up.railway.app
Swagger (demo)	telco-api-production-e466.up.railway.app/docs
Base de datos	PostgreSQL 17 en Supabase (proyecto telco-churn)

8.2 Evidencia de ejecución (extracto de logs)

```
ingesta      | Ingesta completada | filas=7043 | columnas=21
limpieza     | TotalCharges: 11 NaN imputados con 0 (tenure=0)
validacion   | estructural: 7043 ok, 0 rechazados
validacion   | semantica: 7043 ok, 0 rechazados
carga_bd     | Tablas de estado vaciadas (full-refresh)
carga_bd     | Insertados 7043 registros en clientes
orquestador  | FIN PIPELINE | duracion total = ~2 seg
```

Con un dataset que contiene errores intencionales (`scripts/inyectar_errores.py`), el pipeline los detecta y separa: **5 estructurales + 3 semánticos** rechazados y auditados, demostrando el control de calidad. Validado además con `pytest` (6/6) en CI y pruebas end-to-end en producción (idempotencia confirmada: reejecutar deja siempre 7.043 registros, 0 duplicados).

8.3 Modelo de datos



El modelo es de **tabla única analítica** (clientes): cada fila es un cliente con sus atributos planos, sin claves foráneas entre entidades. La integridad se garantiza con **clave primaria** (customer_id), **restricciones CHECK** (rangos y dominios), **NOT NULL** y **transacciones**. carga_logs y clientes_rechazados son tablas de auditoría independiente (relación lógica por fecha, no por FK), lo que simplifica las recargas full-refresh.

9. Conclusiones y próximos pasos

La solución cumple los cuatro requisitos del pipeline DataOps —ingesta, limpieza, validación y carga— de forma **reproducible** (Docker + dataset versionado), **trazable** (logs y auditoría en BD), **segura** (SSL, control de acceso, secretos fuera del repo) y **escalable por módulos** (arquitectura cloud desacoplada). El sistema está desplegado y probado end-to-end en producción.

Próximos pasos: (Evaluación 3) entrenar un modelo de clasificación binaria sobre churn usando las variables de mayor poder discriminante (tenure, Contract, MonthlyCharges, InternetService, PaymentMethod); a mediano plazo, conectar una fuente real vía la etapa de ingest, migrar la orquestación a Airflow ante múltiples fuentes y exponer un dashboard de KPIs en Grafana o Power BI.

Anexo A — Evidencias de ejecución

El cuerpo del informe corresponde a las secciones 1 a 9 (páginas 1–11). Este anexo es material de respaldo: capturas reales del sistema en producción, tomadas el 29 de mayo de 2026.

A.1 — API documentada (Swagger UI en producción)

Telco Churn DataOps API 1.0.0 OAS 3.1

/openapi.json

Pipeline DataOps desacoplado para preparacion de datos de churn. Cada etapa del pipeline es un endpoint independiente. Postgres provisto por Supabase, deploy en Railway.

info

GET	/	Root	⌵
GET	/health	Health	⌵

pipeline

POST	/pipeline/ingest	Endpoint Ingest	⌵
POST	/pipeline/clean	Endpoint Clean	⌵
POST	/pipeline/validate	Endpoint Validate	⌵
POST	/pipeline/load	Endpoint Load	⌵
POST	/pipeline/run	Endpoint Run Pipeline	⌵

monitoreo

GET	/kpis/last	Kpis Ultimos	⌵
GET	/kpis/resumen	Kpis Resumen	⌵
GET	/logs/last	Logs Ultimos	⌵

Endpoints REST del pipeline expuestos y autodocumentados en /docs.

A.2 — Pipeline ejecutado en vivo (POST /pipeline/run)

https://telco-api-production-e466.up.railway.app/pipeline/run \

```
-H 'accept: application/json' \
-d ''
```

Request URL

https://telco-api-production-e466.up.railway.app/pipeline/run

Server response

Code	Details						
200	Response body <pre>{ "estado": "OK", "duracion_total_segundos": 2.126, "etapas": [{ "etapa": "ingesta", "estado": "OK", "duracion_segundos": 0.022, "detalles": { "archivo_salida": "telco_churn_raw_20260529_200549.csv" } }, { "etapa": "limpieza", "estado": "OK", "duracion_segundos": 0.047, "detalles": { "archivo_salida": "telco_churn_clean_20260529_200549.csv" } }, { "etapa": "validacion", "estado": "OK", "duracion_segundos": 0.175, "detalles": { "archivo_validados": "telco_churn_valid_20260529_200549.csv", "archivo_rechazados": null } }] }</pre> Response headers <pre>access-control-allow-origin: * content-length: 680 content-type: application/json date: Fri, 29 May 2025 20:05:48 GMT server: railway-edge x-railway-edge: railway/us-east4-eqdc4a x-railway-request-id: p2uswPATQ02grseiqmzx2A</pre> Responses <table><thead><tr><th>Code</th><th>Description</th><th>Links</th></tr></thead><tbody><tr><td>200</td><td>Successful Response</td><td>No links</td></tr></tbody></table> Media type	Code	Description	Links	200	Successful Response	No links
Code	Description	Links					
200	Successful Response	No links					

Respuesta 200 con las 4 etapas OK y archivos generados; URL real de Railway.

A.3 — Datos en Supabase (consulta SQL directa)

Evidencia — Datos en Supabase PostgreSQL 17

Consulta SQL directa a la base de datos en producción · proyecto telco-churn · PostgreSQL 17.6 on aarch64-unknown-linux-gnu

7.043

clientes cargados

5174

No churn

1869

Churn (abandono)

11

ejecuciones auditadas (carga_logs)

```
SELECT customer_id, gender, tenure, contract, monthly_charges, total_charges, tenure_group, churn
FROM clientes ORDER BY customer_id LIMIT 6;
```

customer_id	gender	tenure	contract	monthly_charges	total_charges	tenure_group	churn
0002-ORFBO	Female	9	One year	65.60	593.30	0-12	false
0003-MKNFE	Male	9	Month-to-month	59.90	542.40	0-12	false
0004-TLHLJ	Male	4	Month-to-month	73.90	280.85	0-12	true
0011-IGKFF	Male	13	Month-to-month	98.00	1237.85	13-24	true
0013-EXCHZ	Female	3	Month-to-month	83.90	267.40	0-12	true
0013-MHZWF	Female	9	Month-to-month	69.40	571.45	0-12	false

Última carga (tabla carga_logs): insertados=7043, rechazados=0, estado=OK, duración=1.734s

7.043 clientes cargados en PostgreSQL 17; muestra real de la tabla clientes.

A.4 — Integración continua (GitHub Actions)

Platform Solutions Resources Open Source Enterprise Pricing

Search Sign in Sign up

BenjaminHeresmann / telco-churn-pipeline Public

Notifications Fork 0 Star 0

<> Code Issues Pull requests Actions Projects Security and quality Insights

Actions

All workflows

CI - Tests y validacion

Management

Caches

All workflows

Showing runs from all workflows

Filter workflow runs

8 workflow runs

Workflow	Event	Status	Branch	Actor
Informe tecnico en .docx + PDF (11 pags, formato rub...	main	12 minutes ago	...	
CI - Tests y validacion #8: Commit 1fb687a pushed by BenjaminHeresmann				
Auditoria docs: corregir 7032->7043, PostgreSQL 17, ...	main	39 minutes ago	...	
CI - Tests y validacion #7: Commit e61dc54 pushed by BenjaminHeresmann				
Auditoria: engine singleton (pool reuse), fix empareja...	main	Today at 2:59 PM	...	
CI - Tests y validacion #6: Commit e138635 pushed by BenjaminHeresmann				
Docs: agregar URLs de produccion (Railway + Supaba...	main	Today at 2:37 PM	...	
CI - Tests y validacion #5: Commit 4e7e580 pushed by BenjaminHeresmann				
Carga idempotente (full-refresh) + fix railway.toml st...	main	Today at 2:34 PM	...	
CI - Tests y validacion #4: Commit 0ac8c20 pushed by BenjaminHeresmann				
Fallback simple: dataset versionado en repo + ingest...	main	Today at 1:37 PM	...	
CI - Tests y validacion #3: Commit 179fe95 pushed by BenjaminHeresmann				
Agregar script de setup automatico de Supabase (DD...	main	Today at 1:15 PM	...	
CI - Tests y validacion #2: Commit c39a0a7 pushed by BenjaminHeresmann				

Workflows de CI en verde para cada push al repositorio público.